

4.10 - Policy-as-code: OPA, Casbin, OpenFGA

Autorização que sai do código da aplicação

Por que externalizar a autorização

De regra espalhada a fonte única de verdade

- **Política espalhada no código é impossível de auditar**
 - Cada serviço decide do próprio jeito, sem padrão
- **Policy-as-code centraliza a regra num componente dedicado**
 - A aplicação pergunta, o motor de política responde
- **Política vira artefato versionado, testável e revisável**
 - Mudança de regra não exige deploy da aplicação
- **Auditoria passa a ter uma fonte única de verdade**

Separar decisão de aplicação da política

- O PEP intercepta o pedido e pergunta se pode
- O PDP avalia a política e devolve a decisão

Esse padrão, Policy Enforcement Point e Policy Decision Point, é o que torna a autorização consistente entre serviços escritos em linguagens diferentes.

- O serviço só precisa saber perguntar, não decidir

Política de autorização em Rego (OPA)

REGO

```
package lunalog.authz

default allow = false

# admin da transportadora gerencia seus motoristas
allow {
  input.user.role == 'admin_transportadora'
  input.user.transportadora == input.resource.transportadora
}

# motorista ve apenas a propria rota do dia
allow {
  input.user.role == 'motorista'
  input.resource.motorista == input.user.id
  input.resource.data == input.now.date
}
```

OPA, Casbin e OpenFGA: quando cada um

- **OPA usa Rego, ótimo para políticas genéricas**
 - Desacopla a decisão de qualquer linguagem de aplicação
- **Casbin suporta vários modelos por configuração declarativa**
 - RBAC, ABAC e mais em uma sintaxe enxuta
- **OpenFGA é motor ReBAC inspirado no Zanzibar**
 - Ideal para relacionamentos e compartilhamento em grafo
- **A escolha segue o modelo de permissão do produto**

Autorização embutida ou externalizada

Embutida no código

- Regra acoplada à lógica de negócio
- Inconsistente entre serviços e linguagens
- Mudança exige novo deploy
- Auditoria precisa ler todo o código

Externalizada como política

- Regra separada e declarativa
- Consistente para todos os serviços
- Mudança sem novo deploy da aplicação
- Auditoria lê uma fonte única

Doze serviços, doze autorizações diferentes

- **Auditoria encontra políticas inconsistentes entre serviços**
 - Um usa enum, outro tabela, outro if aninhado

LUNALOG

Com doze microsserviços em produção, a LunaLog tinha doze implementações diferentes de autorização: um time usava enum hardcoded, outro guardava regras em tabela no banco, outro tinha um if aninhado de oitenta linhas. Uma auditoria encontrou inconsistências reais nas políticas entre serviços, regras que diziam coisas diferentes para o mesmo caso. A equipe adotou OPA para centralizar todas as políticas num lugar versionável, testável e consistente. A partir dali, mudar uma regra de autorização deixou de exigir o deploy de doze serviços. Ana finalmente conseguia responder com precisão quem podia fazer o quê na plataforma.



Suas políticas agora são código, versionadas e testáveis. Mas existe um limite: política só protege contra a ameaça que você já conhece. E as ameaças que ninguém mapeou ainda? Existe uma forma de encontrá-las de propósito, antes que um atacante as encontre por acidente.

Threat modeling: STRIDE e árvores de ataque